CARDIOAI

Inteligentny system wspomagający analizę 12-odprowadzeniowych sygnałów EKG

Specyfikacja projektowo-badawcza

Uniwersytet Bielsko-Bialski
Wydział Budowy Maszyn i Informatyki
Kierunek: Informatyka | Przedmiot: Uczenie maszynowe
Zespół projektowy: 3 osoby
Zbiór danych: PTB-XL v1.0.3 (PhysioNet)

Dokument przeznaczony do uzgodnienia zakresu z prowadzącym oraz jako wewnętrzna specyfikacja implementacyjna zespołu.

Streszczenie

Celem projektu jest zaprojektowanie i implementacja kompletnego systemu uczenia maszynowego do automatycznej analizy 12-odprowadzeniowych zapisów EKG. System będzie rozwijany na publicznym zbiorze PTB-XL w wersji 1.0.3 i obejmie pełny cykl inżynierii ML: pozyskanie i wersjonowanie danych, przygotowanie etykiet, walidację jakości sygnału, preprocessing, trening i porównanie architektur 1D, kalibrację prawdopodobieństw, mechanizm abstention, explainability, eksperymenty odpornościowe, śledzenie eksperymentów, API inferencyjne, interfejs webowy i deployment kontenerowy.

Rdzeniem problemu będzie wieloetykietowa klasyfikacja pięciu głównych klas diagnostycznych PTB-XL: NORM, MI, STTC, CD i HYP. Projekt nie ma charakteru urządzenia medycznego i nie będzie służył do rzeczywistego diagnozowania pacjentów. Wynik należy traktować jako prototyp badawczo-inżynierski systemu wspomagania interpretacji EKG oraz platformę eksperymentalną do oceny architektur, kalibracji i odporności modeli.

Założenie nadrzędne: wartość projektu ma wynikać nie z pojedynczego wyniku AUC, lecz z jakości kompletnego pipeline'u: poprawnego patient-wise splitu, powtarzalnych eksperymentów, kontroli jakości wejścia, probabilistycznej wiarygodności predykcji i możliwości wdrożenia modelu jako usługi.

Zakres dokumentu

- 1. Cel, motywacja i zakres projektu
- 2. Zbiór PTB-XL 1.0.3 i formalizacja problemu
- 3. Architektura systemu
- 4. Pipeline danych i preprocessing
- 5. Architektury modeli i strategia uczenia
- 6. Quality Control, robustness, kalibracja i XAI
- 7. Protokół eksperymentalny i metryki
- 8. Stack technologiczny i organizacja repozytorium
- 9. API, frontend i deployment
- 10. Strategia wykorzystania sprzętu
- 11. Podział odpowiedzialności w zespole
- 12. Harmonogram realizacji
- 13. Ryzyka, kryteria akceptacji i rozszerzenia
- 14. Literatura i źródła

1. Cel, motywacja i zakres projektu

Projekt ma odzwierciedlać rzeczywisty proces wytwarzania systemu ML, a nie wyłącznie trening modelu w notebooku. Przedmiotem implementacji będzie system przetwarzający kompletny 10-sekundowy zapis 12-odprowadzeniowego EKG i zwracający predykcje wieloetykietowe wraz z miarą pewności, informacją o jakości sygnału oraz wizualizacją fragmentów sygnału wpływających na decyzję modelu.

1.1. Cele główne

- Zbudowanie reprodukowalnego pipeline'u danych dla PTB-XL 1.0.3 z zachowaniem oficjalnego patient-wise splitu.
- Implementacja i porównanie co najmniej trzech rodzin modeli: prosty CNN 1D jako baseline, ResNet1D jako model główny oraz kompaktowy Transformer/temporal encoder jako model porównawczy.
- Wielozadaniowe rozszerzenie modelu o ocenę jakości sygnału oraz - opcjonalnie - pomocnicze głowy rhythm/form.
- Kalibracja predykcji i zaprojektowanie mechanizmu abstention, tak aby system mógł jawnie odmówić wydania wyniku w przypadku słabej jakości wejścia lub wysokiej niepewności.
- Ocena odporności na zakłócenia, brakujące odprowadzenia i zmianę częstotliwości próbkowania.
- Implementacja warstwy XAI pozwalającej analizować wkład odprowadzeń i fragmentów czasowych w predykcję.
- Zamknięcie rozwiązania w pełny produkt demonstracyjny: backend FastAPI, frontend React/TypeScript, śledzenie eksperymentów MLflow, CI oraz konteneryzacja deploymentu.

1.2. Świadome ograniczenia zakresu

- System nie będzie certyfikowanym wyrobem medycznym i nie może być wykorzystywany do podejmowania decyzji klinicznych.
- Nie zakładamy trenowania dużego foundation modelu od zera. Priorytetem jest jakość eksperymentu, architektury i oprogramowania przy dostępnych zasobach obliczeniowych.
- Nie będziemy tworzyć sztucznego problemu obrazu EKG. Dane są sygnałem wielokanałowym i podstawowy pipeline będzie operował bezpośrednio na reprezentacji 1D.
- Wersja 100 Hz będzie domyślnym wariantem rozwojowym; 500 Hz zostanie wykorzystane do wybranych badań porównawczych po ustabilizowaniu pipeline'u.

1.3. Hipotezy badawcze

ID	Hipoteza / pytanie badawcze
H1	Architektura residual 1D osiągnie wyższą jakość i stabilność niż płytki CNN przy zachowaniu akceptowalnego kosztu inferencji.
H2	Fuzja wybranych metadanych demograficznych może poprawić kalibrację i rozpoznawanie części klas, ale wymaga kontroli ryzyka shortcut learning.
H3	Controlled lead dropout podczas treningu zwiększy odporność na brak pojedynczych odprowadzeń bez istotnego spadku jakości na pełnym 12-lead ECG.
H4	Kalibracja post-hoc i mechanizm abstention zwiększą wiarygodność systemu przy ograniczeniu coverage.
H5	Wzrost częstotliwości z 100 Hz do 500 Hz nie musi proporcjonalnie zwiększać jakości klasyfikacji w relacji do 5x większego kosztu obliczeniowego.
H6	Model uczony wielozadaniowo z pomocniczym zadaniem jakości sygnału może wykazywać większą odporność na artefakty niż model czysto diagnostyczny.

2. Zbiór PTB-XL 1.0.3 i formalizacja problemu

PTB-XL 1.0.3 jest otwartym zbiorem klinicznych EKG opublikowanym w PhysioNet. Wersja 1.0.3 zawiera 21 799 rekordów 12-odprowadzeniowego EKG pochodzących od 18 869 pacjentów. Każdy rekord ma długość 10 s. Dane falowe

są udostępnione w formacie WFDB zarówno przy 500 Hz, jak i w wersji downsampled 100 Hz. Etykiety bazują na standardzie SCP-ECG i obejmują 71 stwierdzeń diagnostycznych, morfologicznych i rytmicznych. Zbiór zawiera również metadane demograficzne, opis jakości sygnału oraz oficjalne foldy do walidacji krzyżowej [1,2].

Właściwość	Wartość / znaczenie projektowe
Wersja	1.0.3, publikacja PhysioNet: 9 listopada 2022
Liczba rekordów	21 799 zapisów klinicznych ECG
Liczba pacjentów	18 869; konieczny split patient-wise
Długość	10 s na rekord
Kanały	12 standardowych odprowadzeń: I, II, III, aVR, aVL, aVF, V1-V6
Sampling	100 Hz i 500 Hz
Format	WFDB, 16-bit; metadane w ptbxl_database.csv
Adnotacje	71 SCP-ECG statements; kategorie diagnostic / form / rhythm
Jakość sygnału	static_noise, burst_noise, baseline_drift, electrodes_problems, extra_beats, pacemaker
Licencja	Creative Commons Attribution 4.0
Rozmiar	ok. 3.0 GB po rozpakowaniu; ZIP ok. 1.7 GB

2.1. Etykiety głównego zadania

Pierwszym poziomem projektu będzie pięć klas diagnostycznych agregowanych przez PTB-XL do superklas. Ze względu na współwystępowanie patologii zadanie będzie traktowane jako klasyfikacja multi-label; wyjście modelu ma postać pięciu niezależnych logitów z aktywacją sigmoid.

Kod	Znaczenie	Liczba rekordów w PTB-XL 1.0.3
NORM	Normal ECG	9 514
MI	Myocardial Infarction	5 469
STTC	ST/T Change	5 235
CD	Conduction Disturbance	4 898
HYP	Hypertrophy	2 649

Suma liczebności klas przekracza liczbę rekordów, ponieważ jeden zapis może posiadać więcej niż jedną etykietę. W kolejnej iteracji model może zostać rozszerzony na diagnostic subclasses albo na wybrane szczegółowe statements o dostatecznej liczebności.

2.2. Split danych i ochrona przed leakage

Projekt bezwzględnie wykorzystuje kolumnę strat_fold dostarczoną przez autorów zbioru. Wszystkie rekordy danego pacjenta znajdują się w tym samym foldzie, co chroni przed zawyżeniem wyniku przez obecność zapisów jednej osoby zarówno w train, jak i test. Autorzy rekomendują foldy 1-8 do treningu, fold 9 do walidacji i fold 10 do testu; rekordy w foldach 9 i 10 przeszły co najmniej jedną ocenę człowieka i mają szczególnie wysoką jakość etykiet [1].

```
TRAIN: strat_fold in {1,2,3,4,5,6,7,8}
VALIDATION: strat_fold == 9
TEST: strat_fold == 10

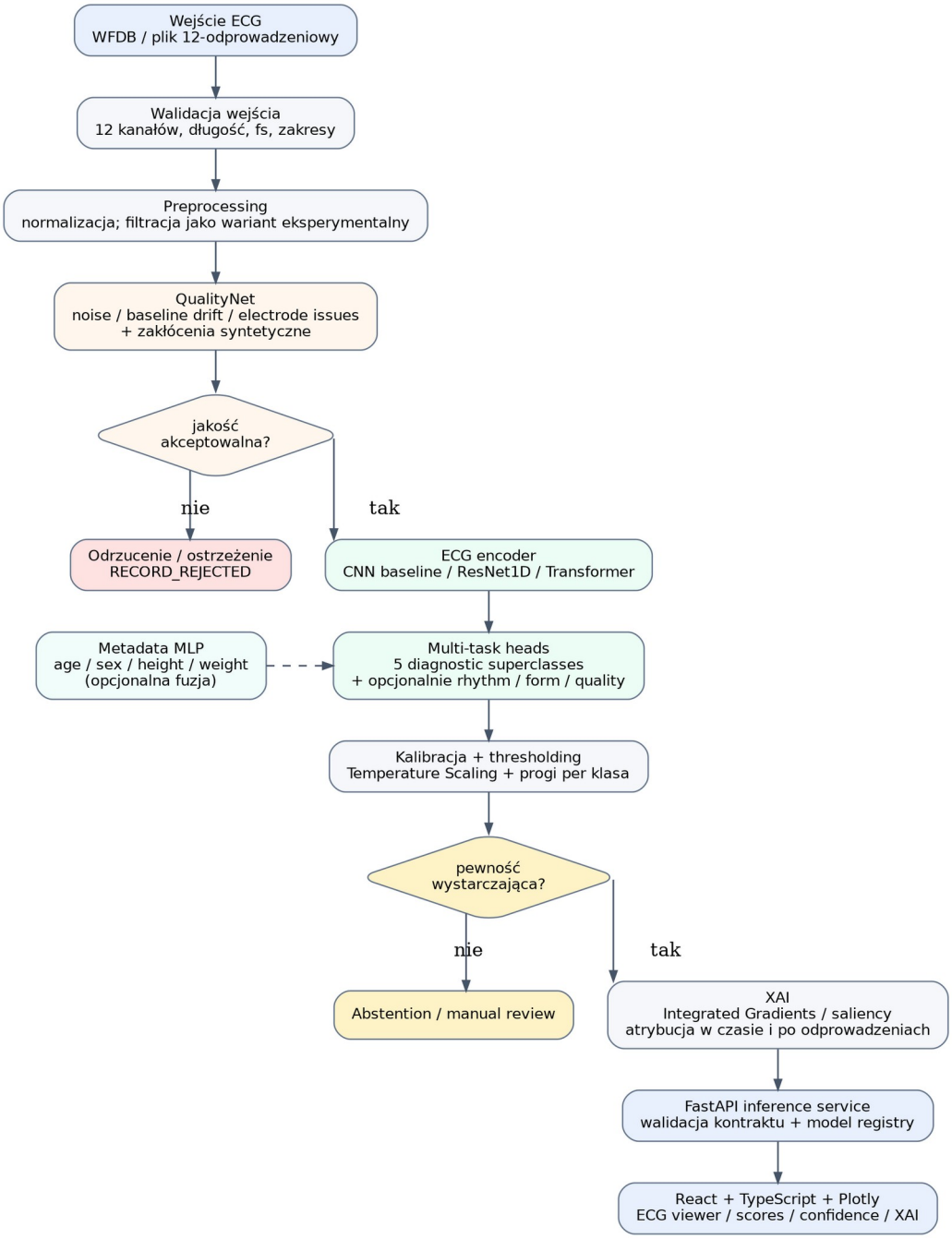
Zasada: test set jest "zamrożony". Nie służy do doboru architektury, progów ani
hiperparametrów.
```

2.3. Znaczenie wersji 1.0.3

Wersja 1.0.3 usuwa dwa kolejne duplikaty (dawne triplikaty) oraz stosuje etykiety konsensusowe dla duplikatów; wersja 1.0.2 wcześniej usunęła 36 rekordów o identycznych surowych przebiegach. W związku z tym wyniki historycznych benchmarków na starszych rewizjach zbioru należy traktować jako punkt odniesienia, a nie jako bitowo porównywalny target [1].

3. Architektura systemu

System zostanie zaprojektowany warstwowo. Kluczowym założeniem jest separacja logiki danych, modelu, warstwy niezawodności oraz produktu inferencyjnego. Umożliwia to równoległą pracę trzech osób, testowanie komponentów izolowanie i późniejsze zastępowanie modeli bez przepisywania API lub UI.



Rysunek 1. Docelowa architektura logiczna systemu CardioAI.

3.1. Przepływ inferencji

1. Walidacja pliku wejściowego: liczba kanałów, częstotliwość próbkowania, długość, brak NaN/Inf, jednostki i zakresy amplitud.

2. Preprocessing zgodny z konfiguracją modelu: normalizacja oparta wyłącznie na statystykach train oraz ewentualny wariant filtracji.
3. Ocena jakości sygnału. W przypadku jakości poniżej progu system zwraca status RECORD_REJECTED / LOW_QUALITY zamiast wymuszać klasyfikację diagnostyczną.
4. Ekstrakcja reprezentacji przez encoder sygnału; opcjonalna fuzja z metadanymi.
5. Predykcja wieloetykietowa oraz kalibracja prawdopodobieństw.
6. Ocena niepewności. Dla przypadków niejednoznacznych aktywowany jest abstention/manual review.
7. Dla zaakceptowanych predykcji generowana jest mapa atrybucji XAI.
8. Backend zwraca ustandaryzowany JSON; frontend wizualizuje 12 odprowadzeń, scores, confidence, ostrzeżenia i XAI.

3.2. Poziomy funkcjonalne

Poziom	Zakres	Status
MVP	PTB-XL loader, 100 Hz, 5 superklas, CNN baseline, ResNet1D, metryki, MLflow	obowiązkowy
Core	QualityNet, calibration, abstention, XAI, robustness, API + UI	obowiązkowy
Extended	Transformer, metadata fusion, 500 Hz, auxiliary heads, ONNX/quantization	realizowany po ustabilizowaniu Core
Research+	hierarchiczne labels, self-supervised pretraining, external validation	opcjonalny / future work

4. Pipeline danych i preprocessing

4.1. Pozyskanie i wersjonowanie

Danych nie należy commitować do Git. Repozytorium będzie zawierało skrypt pobierający konkretnie wersję 1.0.3 z PhysioNet, plik manifestu oraz kontrolę sum SHA-256. Pozwala to każdej osobie odtworzyć identyczne środowisko danych bez Git LFS i bez niepotrzebnego kopiowania 3 GB binariów.

```
data/
  raw/ptb-xl-1.0.3/        # nieversionowane
  interim/                 # cache i indeksy
  processed/               # opcjonalne cache tensorów

scripts/
  download_ptbxl.py
  verify_checksums.py
  build_label_index.py
```

4.2. Ładowanie sygnału

Podstawowym loaderem będzie biblioteka WFDB. Rekord 100 Hz reprezentujemy jako tensor float32 o kształcie [12, 1000], a rekord 500 Hz jako [12, 5000]. W DataLoaderze zwracamy sygnał, wektor 5 etykiet, patient_id, ecg_id, fold oraz wymagane metadane. Indeks danych budujemy raz z ptbxl_database.csv i scp_statements.csv, aby nie parsować słowników etykiet w każdej epoce.

4.3. Minimalny preprocessing jako baseline

- Konwersja do float32 i weryfikacja kształtu.
- Obsługa wyjątków: brakujące kanały, uszkodzone rekordy, wartości nie-skończone.
- Standaryzacja per odprowadzenie z parametrami wyznaczonymi wyłącznie na train; alternatywnie per-record z-score jako osobny eksperyment.

- Brak agresywnego filtrowania w wariancie bazowym. Informacje ST/T mogą być wrażliwe na preprocessing, dlatego filtrację należy traktować jako ablation, a nie domyślną czynność bez walidacji.

4.4. Warianty filtracji do porównania

W kontrolowanym eksperymencie można zestawić raw/minimal preprocessing z filtracją pasmową, np. 0.5-40 Hz, oraz ewentualnym tłumieniem interferencji sieciowej. Filtry muszą być zerofazowe (np. filtfilt), a wynik należy oceniać nie tylko globalnie, ale per klasa - szczególnie STTC. Żaden filtr nie jest stosowany do testu inaczej niż poprzez identyczny, zamrożony pipeline wybrany na validation.

4.5. Augmentacje treningowe

Augmentacja	Cel	Uwagi
Gaussian noise	odporność na szum pomiarowy	małe SNR perturbations; parametry logowane
Baseline wander	odporność na dryft linii bazowej	niska częstotliwość; tylko train
Amplitude scaling	odporność na różnice amplitudy	współczynnik w ograniczonym zakresie
Time shift	niewrażliwość na fazę okna	circular lub padded shift; ostrożnie
Temporal masking	regularizacja	krótkie segmenty, nie niszczyć całej morfologii
Lead dropout	odporność na utratę kanału	losowo 1-2 odprowadzenia; jeden z głównych eksperymentów

Reguła: augmentacje nigdy nie są aktywne dla validation i test. Każda augmentacja musi być odtwarzalna przez seed i jawnie zapisana w konfiguracji runu.

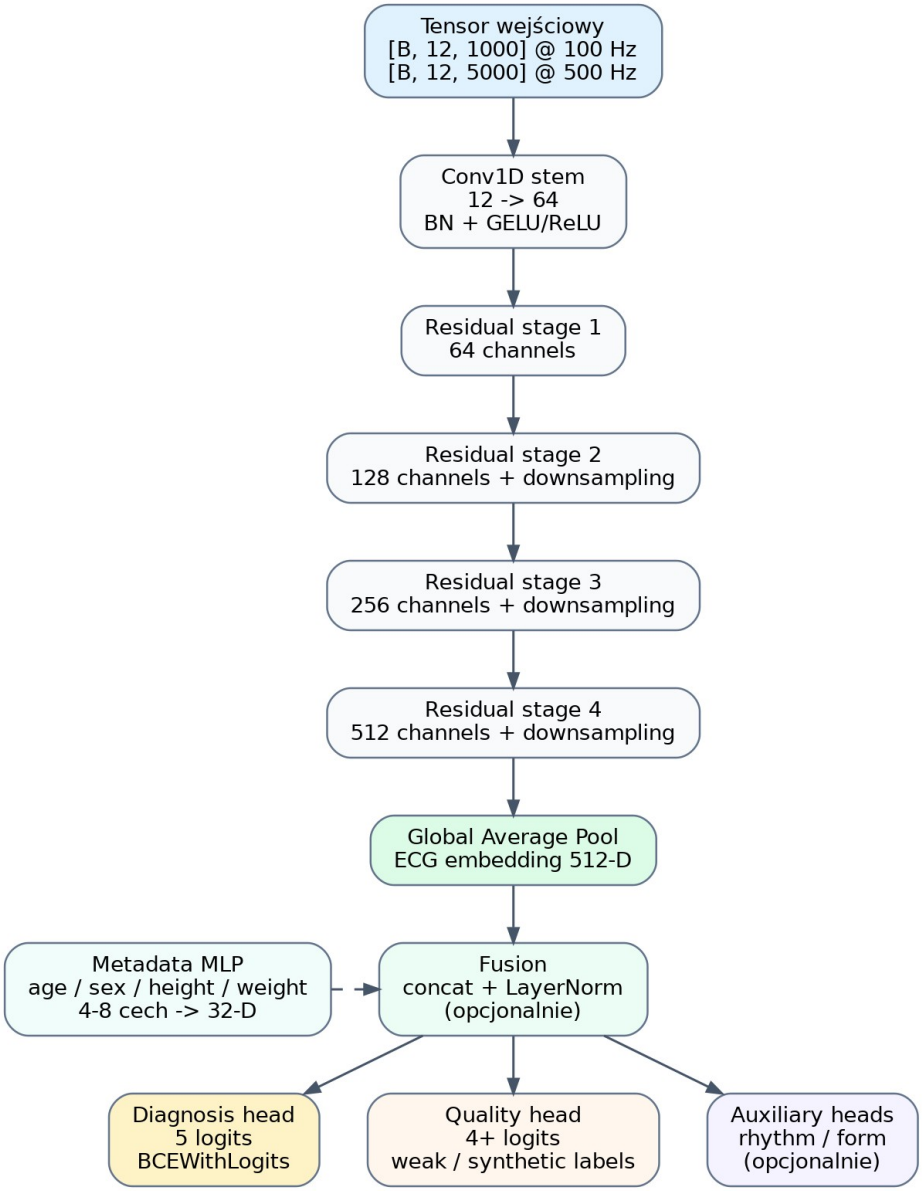
5. Architektury modeli i strategia uczenia

5.1. Baseline klasyczny i płytki CNN

Pierwszy milestone powinien powstać bardzo szybko i stanowić sanity check całego pipeline'u. Rekomendowane są dwa baseline'y: (a) model klasyczny na prostych cechach statystycznych/spektralnych, np. Logistic Regression albo LightGBM, oraz (b) niewielki CNN 1D. Ich rolą nie jest maksymalizacja wyniku, lecz weryfikacja etykiet, splitu, metryk i obsługi multi-label.

5.2. Model główny: ResNet1D

Główną architekturą rekomenduje się residual CNN operujący bezpośrednio na 12 kanałach czasowych. Rodzina ResNet1D ma silny punkt odniesienia w benchmarkach PTB-XL: opublikowany benchmark raportuje dla diagnostic superclasses AUC około 0.93 dla ResNet1D/XResNet1D na starszej rewizji danych [3]. Wynik ten jest punktem odniesienia, a nie gwarantowanym targetem dla wersji 1.0.3 i własnego protokołu.



Rysunek 2. Proponowana struktura modelu głównego ResNet1D z opcjonalną fuzją metadanych i głowami pomocniczymi.

Element	Proponowana konfiguracja startowa
Input	[B, 12, 1000] dla 100 Hz
Stem	Conv1D, kernel 15-31, 64 kanały, BN, GELU/ReLU
Residual stages	64 -> 128 -> 256 -> 512 kanałów, stride/downsampling czasowy
Pooling	Global Average Pooling
Embedding	512-D
Diagnosis head	Linear -> 5 logits; BCEWithLogitsLoss
Metadata branch	MLP -> 32-D, opcjonalnie concat z embeddingiem
Quality head	multi-label; weak labels + synthetic corruption labels
Regularization	dropout, weight decay, augmentacje sygnałowe

5.3. Model porównawczy: Transformer czasowy

Transformer nie powinien być celem samym w sobie. Ma odpowiedzieć, czy explicit long-range attention daje korzyść przy 10-sekundowym EKG względem silnej konwolucyjnej bazy. Sygnał można dzielić na temporal patches, projektować

liniowo do d_model i przetwarzać 4-6 warstwami TransformerEncoder. Z uwagi na pamięć zaleca się stosowanie 100 Hz oraz umiarkowanej liczby tokenów. Model powinien mieć zbliżony rząd wielkości parametrów do ResNet1D, aby porównanie było uczciwe.

5.4. Fuzja metadanych

PTB-XL udostępnia m.in. age, sex, height i weight. Metadane można zakodować małym MLP i połączyć z embeddingiem ECG. Eksperyment musi jednak uwzględnić ryzyko shortcut learning: wzrost AUC nie oznacza automatycznie lepszego rozumienia sygnału. Wynik ECG-only i ECG+metadata należy raportować osobno, a brakujące dane kodować jawnie (masking / missing indicators), a nie zerować bez informacji.

5.5. Funkcja straty i class imbalance

Bazową funkcją będzie BCEWithLogitsLoss dla pięciu niezależnych etykiet. Nierównowagę klas należy obsłużyć eksperymentalnie: pos_weight obliczone wyłącznie na train, focal loss jako wariant oraz threshold tuning na validation. Preferujemy prosty loss + kalibrację i progi per klasa, zanim wprowadzimy złożone funkcje kosztu.

```
loss_total = L_diagnosis
           + lambda_quality * L_quality
           + lambda_aux * L_auxiliary

Wagi lambda są hiperparametrami dobieranymi na foldzie 9, nie na teście.
```

5.6. Proponowany protokół treningu

Parametr	Startowa konfiguracja / zasada
Optimizer	AdamW
Learning rate	np. 3e-4; sweep w ograniczonym zakresie
Scheduler	Cosine annealing lub ReduceLROnPlateau
Batch size	dobierany do urządzenia; gradient accumulation przy ograniczonym VRAM
Precision	float32 jako referencja; AMP na CUDA, o ile stabilne
Early stopping	validation macro-AUROC / brak poprawy przez ustaloną liczbę epok
Checkpoint	najlepszy validation score + stan optimizera + config
Random seeds	minimum 3 seedy dla modelu finalnego
Test	wyłącznie po zamrożeniu architektury i progów

6. Quality Control, robustness, kalibracja i XAI

6.1. Moduł QualityNet

PTB-XL udostępnia pola static_noise, burst_noise, baseline_drift i electrodes_problems. Są to użyteczne metadane jakościowe, lecz nie należy automatycznie traktować ich jako perfekcyjnych etykiet typu clean/bad. Proponujemy podejście hybrydowe: metadata-derived weak labels połączone z kontrolowanymi syntetycznymi korupcjami, dla których ground truth jest znany.

Źródło supervision	Przykład	Rola
PTB-XL metadata	baseline_drift != empty	weak real-world supervision
Synthetic corruption	dodany sinus niskiej częstotliwości	precyzyjna etykieta artefaktu
Lead dropout	wyzerowane V2	test odporności / electrode issue proxy
Noise injection	kontrolowany Gaussian noise	krzywa performance vs SNR

6.2. Mechanizm gate

QualityNet nie powinien od razu blokować wszystkich nieidealnych zapisów. Potrzebne są co najmniej trzy statusy: ACCEPT, WARN oraz REJECT. Progi jakości będą dobierane na validation. W raporcie należy pokazać trade-off: ile rekordów system odrzuca i jak zmienia się jakość diagnozy na pozostałych przypadkach.

6.3. Kalibracja

AUC mierzy ranking, ale nie mówi, czy 0.8 można interpretować jako wiarygodne prawdopodobieństwo. Dlatego po zamrożeniu modelu dopasujemy kalibrator na foldzie 9. Bazową metodą będzie Temperature Scaling; dla multi-label można stosować parametr globalny albo per-class. Alternatywą jest isotonic regression, jeżeli liczebność per klasa będzie wystarczająca.

Raportujemy Brier Score i Expected Calibration Error obok AUROC/F1. Wykres reliability diagram powinien być generowany per klasa i dla modelu przed/po kalibracji.

6.4. Abstention / selective prediction

System otrzyma prawo do jawnego „nie wiem”. Mechanizm może bazować na maksymalnej entropii, odległości prawdopodobieństwa od progów decyzyjnych oraz quality score. Kluczową analizą będzie krzywa coverage-risk: jak spada błąd, gdy system odrzuca najbardziej niepewne 5%, 10%, 20% przypadków.

6.5. Explainable AI

Dla modeli 1D rekomendowane są Integrated Gradients i proste saliency/gradient x input. Wynik powinien być agregowany w dwóch wymiarach: contribution per lead oraz attribution over time. Frontend ma umożliwiać nałożenie atrybucji na przebieg, a także wyświetlenie rankingów odprowadzeń. XAI nie będzie przedstawiane jako kliniczne wyjaśnienie przyczynowe - jedynie jako analiza wrażliwości modelu.

6.6. Robustness suite

Test	Procedura	Raportowana miara
Missing lead	usuwać kolejno każde z 12 odprowadzeń	delta AUROC/F1 per lead
Random lead dropout	1-2 brakujące leads	quality vs number of missing leads
Noise sweep	kilka poziomów kontrolowanego SNR	degradation curve
Baseline drift	kontrolowana amplituda i częstotliwość	degradation curve
100 vs 500 Hz	ten sam model family / protokół	quality vs compute cost
Preprocessing shift	raw vs filtered	per-class impact

7. Protokół eksperymentalny i metryki

7.1. Kolejność eksperymentów

Etap	Eksperyment	Cel decyzyjny
E0	EDA + sanity checks	potwierdzić liczebności, split, brak leakage
E1	classical baseline	zweryfikować etykiety i metryki
E2	small CNN 1D	pierwszy end-to-end deep baseline
E3	ResNet1D	główny model referencyjny
E4	class weighting / focal / thresholds	obsługa imbalance
E5	quality multitask	wpływ quality supervision
E6	lead dropout / robustness	odporność kanałowa
E7	Transformer	porównanie indukcyjnych biasów
E8	metadata fusion	wartość danych pozasygnałowych
E9	calibration + abstention	wiarygodność predykcji
E10	100 vs 500 Hz	koszt vs jakość
E11	XAI + deployment benchmark	gotowość systemowa

7.2. Metryki klasyfikacji

- Macro AUROC - główna metryka porównania z literaturą i między klasami.
- Micro AUROC - obraz globalny zdominowany przez częstsze etykiety.
- Macro F1 i per-class F1 po ustaleniu progów na validation.
- Sensitivity/Recall i Specificity per klasa.
- Average Precision / PR-AUC, szczególnie dla rzadszych klas.
- Confusion-like analysis dla multi-label: TP/FP/FN per klasa i analiza błędów klinicznie podobnych kategorii.

7.3. Metryki niezawodności i systemu

Kategoria	Metryki
Calibration	Brier score, ECE, reliability diagrams
Selective prediction	coverage, selective risk, AUROC na accepted subset
Quality	AUROC/F1 dla typów artefaktów, false reject rate
Robustness	delta metric względem clean baseline
Runtime	latency p50/p95, throughput, peak RAM/VRAM, rozmiar artefaktu
Reproducibility	średnia ± odchylenie dla finalnych seedów

7.4. Benchmark referencyjny

Repozytorium autorów benchmarku PTB-XL raportuje dla diagnostic superclasses AUC ~0.930 dla ResNet1D oraz ~0.928 dla XResNet1D, używając foldów 1-8/9/10 [3]. Nasz projekt nie powinien deklarować konieczności pobicia benchmarku - celem jest odtworzenie porównywalnego poziomu przy poprawnym pipeline i rozszerzenie o reliability, quality i deployment. Wyniki mogą różnić się ze względu na rewizję 1.0.3, konfigurację preprocessingu i własną implementację.

8. Stack technologiczny i organizacja repozytorium

8.1. Stack

Warstwa	Technologia	Uzasadnienie
Język	Python 3.11/3.12	dominujący ekosystem ML i obsługa wszystkich wymaganych bibliotek
ECG I/O	WFDB	natywny format PTB-XL; metadane nagłówków i sygnał
Data	NumPy, Pandas, SciPy	indeksowanie, statystyka i processing sygnałowy
ML klasyczny	scikit-learn	baseline, calibration, metryki pomocnicze
Deep Learning	PyTorch	CUDA na RTX; MPS na Apple Silicon [4]
Metryki	torchmetrics + scikit-learn	spójne metryki batch/epoch i finalna ewaluacja
XAI	Captum lub własne Integrated Gradients	atrybucja dla PyTorch
Tracking	MLflow	runs, parametry, metryki, artefakty i model registry [5]
Backend	FastAPI + Pydantic	typowany kontrakt inferencji i OpenAPI
Frontend	React + TypeScript + Plotly	interaktywna wizualizacja 12 kanałów i overlay XAI
Packaging	pyproject.toml + uv/pip	reprodukowalne środowisko; osobne instrukcje CUDA/MPS
Quality	pytest, ruff, mypy opcjonalnie	automatyczne testy i spójność kodu
CI/CD	GitHub Actions	testy, lint, build obrazu, artefakty release
Deployment	Docker + Compose	powtarzalny demo deployment
Inference opt.	ONNX Runtime opcjonalnie	CPU portability i możliwość INT8; static quantization jest rekomendowana dla CNN [6]

8.2. Dlaczego PyTorch przy heterogenicznym sprzęcie

PyTorch obsługuje zarówno CUDA, jak i backend MPS dla Apple Silicon. Kod modelu musi używać jednego abstrakcyjnego wyboru device i nie może zawierać założeń CUDA-only. Dokumentacja PyTorch wskazuje, że MPS mapuje operacje na Metal Performance Shaders; jednocześnie należy uwzględnić różnice funkcjonalne, np. brak float64 na MPS [4]. W praktyce pipeline powinien bazować na float32, a mixed precision traktować jako optymalizację per urządzenie.

8.3. Proponowana struktura repozytorium

```
cardioai/
├── pyproject.toml
├── README.md
├── configs/
│   ├── data/
│   ├── model/
│   └── experiment/
├── scripts/
│   ├── download_ptbxl.py
│   ├── build_index.py
│   ├── train.py
│   ├── evaluate.py
│   └── export_model.py
├── src/cardioai/
│   ├── data/          # WFDB loader, labels, preprocessing, augmentations
│   ├── models/        # CNN, ResNet1D, Transformer, heads
│   ├── training/      # losses, loops, metrics, checkpoints
│   ├── reliability/   # calibration, abstention, XAI, robustness
│   ├── inference/     # preprocessing contract + model wrapper
│   └── utils/
├── api/               # FastAPI
├── web/               # React/TypeScript
├── tests/
├── docker/
└── docs/
    ├── experiments.md
    └── model_card.md
```

8.4. Konfiguracja eksperymentów

Hiperparametry nie powinny być wpisane na stałe w kodzie. Każdy run otrzymuje jawny plik YAML/TOML: dataset_version, sampling_rate, normalization, augmentations, architecture, optimizer, seed, loss i progi. MLflow zapisuje config jako artefakt razem z commit SHA. Dzięki temu wynik można przypisać do konkretnego stanu repozytorium i danych.

9. API, frontend i deployment

9.1. Kontrakt API

Warstwa inferencyjna powinna być możliwie niezależna od frontendowej. Proponowany minimalny interfejs:

```
POST /v1/analyze
  multipart: wfdb/npz/csv + optional metadata

GET  /v1/models
GET  /v1/health
GET  /v1/version

Response /v1/analyze:
{
  "model_version": "resnet1d-1.3.0",
  "signal_quality": {"status": "ACCEPT", "score": 0.94},
  "predictions": {
      "NORM": 0.03, "MI": 0.86, "STTC": 0.71, "CD": 0.12, "HYP": 0.08
```

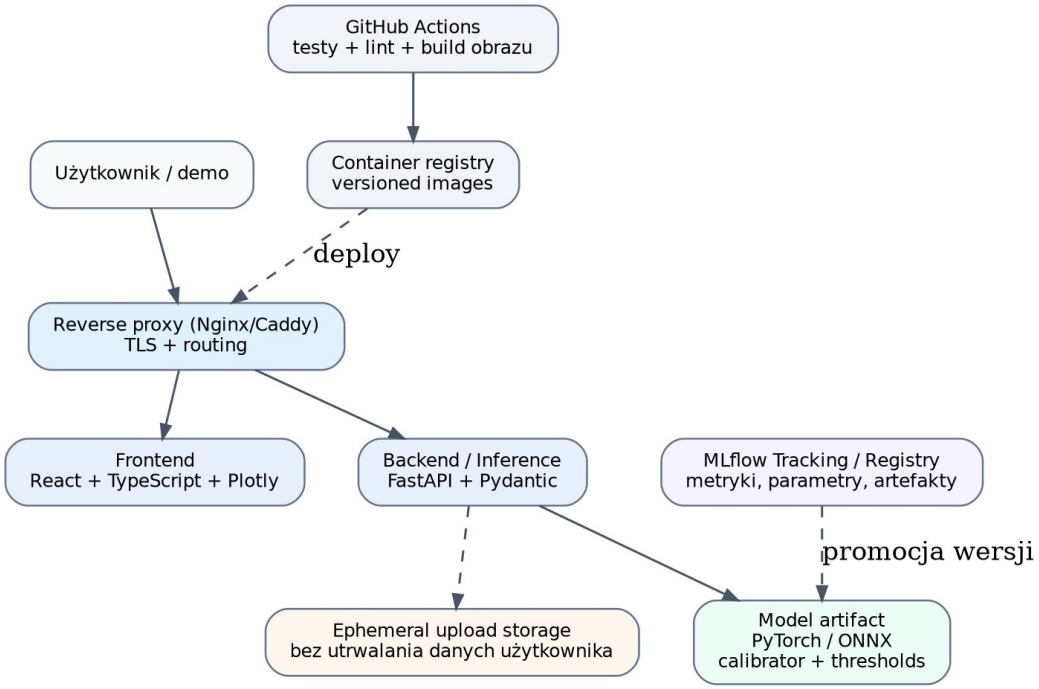
```
    },
    "abstain": false,
    "thresholds": {...},
    "lead_importance": {...},
    "warnings": []
  }
```

Prawdopodobieństwa w przykładzie są ilustracyjne. API powinno jawnie zwracać wersję modelu, wersję preprocessingu i flagę abstention, aby UI nie mogło interpretować surowych logitów poza kontekstem modelu.

9.2. Frontend

- Widok 12 odprowadzeń ze wspólną osią czasu oraz możliwością zoom/pan.
- Panel jakości sygnału i wykrytych artefaktów.
- Wyniki multi-label wraz z progami decyzyjnymi i confidence.
- Panel XAI: heatmapa czasowa i ranking importance per lead.
- Widok metadata/model info: model version, sampling, preprocessing, runtime.
- Tryb demonstracyjny pozwalający wybierać rekordy PTB-XL po ecg_id bez uploadu danych użytkownika.

9.3. Deployment



Rysunek 3. Proponowana architektura deploymentu i ścieżka artefaktów.

Dla projektu semestralnego rekomendowany jest Docker Compose z osobnymi usługami frontend i API. Model może być ładowany bezpośrednio z wersjonowanego artefaktu lokalnego albo z MLflow Model Registry. Nginx/Caddy pełni funkcję reverse proxy. System domyślnie nie utrwala plików wgrywanych przez użytkownika; dane tymczasowe są kasowane po zakończeniu requestu.

9.4. Export i optymalizacja inferencji

Model referencyjny pozostaje w PyTorch, natomiast po finalizacji można wyeksportować ResNet1D do ONNX. ONNX Runtime wspiera post-training quantization; dokumentacja rekomenduje static quantization szczególnie dla CNN, a

dynamic quantization częściej dla RNN/Transformer [6]. Kwantyzacja jest jednak opcją optymalizacyjną - akceptujemy ją tylko wtedy, gdy walidacja wykazuje kontrolowany spadek jakości.

9.5. CI/CD

Trigger	Pipeline
Pull request	ruff -> unit tests -> data-contract tests -> lightweight model smoke test
Merge main	pełny test suite -> build backend/frontend -> build Docker image
Tag release	build versioned images -> export model card -> deployment demo
Model promotion	validation report -> manual approval -> oznaczenie w registry

10. Strategia wykorzystania dostępnego sprzętu

Zespół dysponuje jednym MacBookiem M4 Pro z 24 GB pamięci zunifikowanej oraz dwoma laptopami klasy RTX 2060 / Intel i5 / 16 GB RAM. Nie rekomenduje się distributed training pomiędzy tymi urządzeniami. Heterogeniczność back-endów zwiększyłaby złożoność szybciej niż skróciła czas treningu. Zamiast tego urządzenia powinny prowadzić równoległe eksperymenty z jednym wspólnym trackerem MLflow.

Sprzęt	Preferowane zadania	Uwagi
MacBook M4 Pro 24 GB	Transformer, większe batche 100 Hz, preprocessing/EDA, API/UI, eksperymenty MPS	PyTorch MPS; float32; natywne środowisko, nie Docker do treningu
RTX 2060 laptop A	ResNet1D, CNN, finalne benchmarki CUDA, AMP	batch size wg VRAM; gradient accumulation
RTX 2060 laptop B	quality model, robustness sweeps, XAI, ONNX benchmark	równoległe eksperymenty, nie kopiować test decisions
CPU wszystkich maszyn	preprocessing, indeksy, klasyczne baseline, testy i inference CPU	cache danych i DataLoader workers dopasowane do 16 GB RAM

10.1. Zasady portability

- Cały kod treningowy musi działać na device=cpu, cuda oraz mps; funkcje nie mogą wywoływać .cuda() bezpośrednio.
- Dtype wejścia: float32 jako wspólny mianownik. AMP jest flagą konfiguracyjną, a nie wymogiem.
- Dane 100 Hz są podstawą większości eksperymentów; 500 Hz uruchamiamy tylko dla wybranych, już ustabilizowanych modeli.
- Cache nie powinien wymagać >16 GB RAM. Preferowane jest lazy loading WFDB albo memmap/NPY shard cache.
- Testy jednostkowe muszą używać małych synthetic tensors, aby CI nie pobierało pełnego PTB-XL.

11. Podział odpowiedzialności w zespole

Podział powinien zapewniać jednoznaczną odpowiedzialność, ale nie tworzyć trzech silosów. Każda osoba jest właścicielem obszaru, natomiast co najmniej jedna druga osoba wykonuje code review i potrafi uruchomić jego pipeline. Integracja odbywa się poprzez ustalone kontrakty danych i API.

Osoba / rola	Odpowiedzialność główna	Deliverables
A - Data & Signal ML	PTB-XL ingestion, labels, preprocessing, augmentacje, QualityNet, data tests	dataset module, EDA, quality report, corruption suite
B - Model & Experiments	CNN/ResNet1D/Transformer, losses, trening, tuning, robustness, final evaluation	model zoo, configs, benchmark table, checkpoints
C - Reliability & Product	calibration, abstention, XAI, MLflow, FastAPI, React, deployment/CI	reliability report, API/UI, Docker/CI, model card

11.1. Wspólne obowiązki

- Definicja eksperymentów i review wyników - decyzje o zmianach modelu nie należą wyłącznie do właściciela modelu.
- PR review: minimum jedna aprobata przed merge do main.
- Wspólne kryteria Definition of Done i jednolite nazewnictwo runów MLflow.
- Każda osoba przygotowuje część dokumentacji technicznej odpowiadającą własnemu obszarowi, ale finalny raport ma jeden styl i jedną terminologię.
- Test fold 10 jest odblokowywany dopiero po formalnym zamrożeniu konfiguracji finalnych modeli.

11.2. RACI dla kluczowych obszarów

Obszar	A	B	C
Data pipeline	R/A	C	C
Model architecture	C	R/A	C
QualityNet	R/A	C	C
Training/evaluation	C	R/A	C
Calibration/XAI	C	C	R/A
API/UI	C	I	R/A
Deployment/CI	C	C	R/A
Final report/demo	R	R	R/A

R = Responsible, A = Accountable, C = Consulted, I = Informed.

12. Harmonogram realizacji

Poniższy plan zakłada 14 tygodni pracy. Krytyczna ścieżka to: poprawny dataset -> baseline -> ResNet1D -> zamrożony protocol -> reliability -> integracja. Frontend nie powinien blokować prac modelowych, a backend może rozwijać się na mockowanych odpowiedziach od pierwszych tygodni.

Tydzień	Kamień milowy	Główne zadania
1	Setup i kontrakty	repo, środowiska, MLflow, downloader 1.0.3, schema danych, Definition of Done
2	EDA + labels	analiza klas, braków, foldów, metadanych jakości, sanity plots
3	Baseline	classical baseline + small CNN, pierwsza pełna ewaluacja fold 9
4	ResNet1D v1	architektura, trening, logging, checkpointing
5	ResNet1D v2	imbalance, progi, tuning; zamknięcie core classifier
6	QualityNet	weak labels, synthetic corruptions, quality gate
7	Robustness	lead dropout, noise/baseline drift, per-lead ablation
8	Transformer	model porównawczy, compute-matched benchmark
9	Metadata + multitask	fuzja i pomocnicze heads; decyzja include/exclude
10	Reliability	calibration, ECE/Brier, abstention, coverage-risk
11	XAI + 500 Hz	atrybucja, eksperyment sampling-rate
12	Product	FastAPI + React, inference contract, demo records
13	Deployment	Docker, ONNX opcjonalnie, CI, runtime benchmark, model card
14	Freeze + final test	jednorazowa finalna ewaluacja fold 10, raport, prezentacja i demo

12.1. Zasada bramek jakościowych

Gate	Warunek przejścia
G1 - Data	reprodukowany split i liczebności; zero patient leakage; loader testowany
G2 - Baseline	model uczy się sensownego sygnału; metryki > losowe; pipeline MLflow działa
G3 - Core model	ResNet1D stabilny na co najmniej 3 runach; brak regresji data pipeline
G4 - Reliability	kalibracja/abstention ocenione; quality gate nie degraduje bez kontroli
G5 - Product	API/UI używa zamrożonego preprocessingu; end-to-end test działa
G6 - Final	konfiguracja zamrożona przed test fold 10; model card i wyniki reprodukowalne

13. Ryzyka, kryteria akceptacji i rozszerzenia

13.1. Rejestr ryzyk

Ryzyko	Skutek	Mitygacja
Leakage między pacjentami	fałszywie wysoki wynik	wyłącznie strat_fold; test automatyczny patient_id disjoint
Zbyt duża złożoność modelu	wolny rozwój / OOM	100 Hz first; kompaktowy ResNet; benchmark compute
Sparse/noisy quality labels	słaby QualityNet	weak supervision + synthetic labels; osobny report limitations
Overfitting do validation	nierealny test score	ograniczony tuning; 3 seedy; freeze przed fold 10
Shortcut przez metadane	niepożądana zależność	ECG-only baseline; ablation metadata; subgroup analysis
XAI nadinterpretowane	fałszywe wnioski	opisywać jako attribution, nie kliniczne wyjaśnienie
MPS/CUDA różnice	niestabilność reprodukcji	device abstraction; canonical final run na jednej klasie urządzenia
Integracja za późno	model działa tylko w notebooku	API mock od tyg. 3; kontrakty wersjonowane

13.2. Definition of Done - wersja końcowa

- Automatyczne pobranie/zweryfikowanie PTB-XL 1.0.3 i odtworzenie splitu 1-8/9/10.
- Co najmniej 3 rodziny modeli porównane w jednym protokole; ResNet1D jako model główny.
- Raport per-class oraz macro metrics na validation i finalnym, jednorazowo użytym test fold 10.
- Co najmniej jedna kompletna analiza robustness: lead dropout + noise/baseline drift.
- Kalibracja i wykres coverage-risk dla abstention.
- XAI widoczne w aplikacji dla wybranego rekordu.
- Model i preprocessing opakowane we wspólny inference artifact z wersją.
- API i UI uruchamiane przez jedno polecenie docker compose up (dla inference/demo).
- MLflow zawiera finalne runy, parametry, metryki i model artifact.
- README, model card, opis ograniczeń oraz instrukcja reprodukcji.

13.3. Kryteria jakościowe zamiast arbitralnych targetów

Nie należy obiecywać z góry konkretnego wyniku klinicznego. Kryterium sukcesu to przede wszystkim porównywalny, reprodukowalny protokół. Jako benchmark orientacyjny można traktować historyczne ~0.93 AUC dla diagnostic superclasses z benchmarku PTB-XL [3]. Realistycznym celem projektowym jest zbliżenie się do poziomu publikowanych modeli przy jednoczesnym dostarczeniu elementów, których standardowy benchmark nie obejmuje: quality gate, calibration, selective prediction, robustness i deployment.

13.4. Rozszerzenia po zakończeniu przedmiotu

- Hierarchiczna klasyfikacja: superclass -> subclass -> SCP statement.
- Self-supervised pretraining na nieetykietowanym sygnale ECG.
- External validation na niezależnym zbiorze ECG w celu pomiaru domain shift.
- Conformal prediction / prediction sets zamiast pojedynczych thresholdów.
- Distillation małego modelu do edge inference.
- Model monitoring: detekcja driftu sygnału i metadanych po wdrożeniu demonstracyjnym.
- Eksport do ONNX/ExecuTorch i pomiar na urządzeniu mobilnym lub edge.

14. Literatura i źródła

[1] Wagner, P., Strodthoff, N., Bousseljot, R.-D., Samek, W., Schaeffter, T. PTB-XL, a large publicly available electrocardiography dataset, version 1.0.3. PhysioNet, 2022. DOI: 10.13026/kfzx-aw45.

[2] Wagner, P., Strodthoff, N., Bousseljot, R.-D., Kreiseler, D., Lunze, F. I., Samek, W., Schaeffter, T. PTB-XL, a large publicly available electrocardiography dataset. Scientific Data 7, 154 (2020). DOI: 10.1038/s41597-020-0495-6.

[3] Strodthoff, N., Wagner, P., Schaeffter, T., Samek, W. Deep Learning for ECG Analysis: Benchmarks and Insights from PTB-XL. IEEE Journal of Biomedical and Health Informatics 25(5), 1519-1528 (2021). DOI: 10.1109/JBHI.2020.3022989. Kod referencyjny: github.com/helme/ecg_ptbxl_benchmarking.

[4] PyTorch Documentation. MPS backend - Metal Performance Shaders backend for macOS/Apple Silicon. docs.pytorch.org, dostęp: 2026-10-02.

[5] MLflow Documentation. ML Experiment Tracking / Tracking Server. mlflow.org/docs, dostęp: 2026-10-02.

[6] ONNX Runtime Documentation. Quantize ONNX models. onnxruntime.ai/docs/performance/model-optimizations/quantization.html, dostęp: 2026-10-02.

Załącznik A. Skrócona propozycja tematu do zatwierdzenia przez prowadzącego

Proponowany tytuł: Projekt i implementacja inteligentnego systemu wspomagającego analizę wielokanałowych sygnałów EKG z wykorzystaniem głębokich sieci neuronowych.

Projekt zakłada opracowanie kompletnego pipeline'u ML dla 12-odprowadzeniowych zapisów EKG na bazie PTB-XL 1.0.3. Zakres obejmuje wieloetykietową klasyfikację głównych kategorii diagnostycznych, porównanie architektur CNN/ResNet/Transformer, automatyczną ocenę jakości sygnału, analizę odporności na zakłócenia i brakujące odprowadzenia, kalibrację probabilistyczną oraz mechanizm abstention. System zostanie wyposażony w moduł explainability oraz wdrożony jako aplikacja webowa z inferencją przez FastAPI. Eksperymenty będą prowadzone zgodnie z oficjalnym patient-wise split PTB-XL oraz śledzone w MLflow. Finalnym rezultatem będzie reprodukowalny prototyp badawczo-inżynierski wraz z raportem eksperymentalnym i demonstratorem.

Załącznik B. Checklist przed finalnym testem

- Czy split pacjentów jest rozłączny i pokrywa dokładnie foldy 1-8/9/10?
- Czy wszystkie statystyki normalizacji zostały policzone wyłącznie na train?
- Czy test fold 10 nie był użyty do doboru architektury, thresholdów ani augmentacji?
- Czy model version, preprocessing version i git SHA są zapisane razem?
- Czy finalny model został uruchomiony dla minimum 3 seedów albo jasno opisano ograniczenie?
- Czy kalibracja została dopasowana wyłącznie na validation?
- Czy XAI jest opisane jako attribution, a nie kliniczne wyjaśnienie?
- Czy model card zawiera ograniczenia danych i zakaz zastosowań klinicznych?
- Czy demo działa z czystego checkoutu zgodnie z README?